

PRÁCTICA 5

CONVERSOR ADC Y GENERADOR DE PWM (para LEDs)

Juan Pedro Bandera Rubio (jpbandera@uma.es)

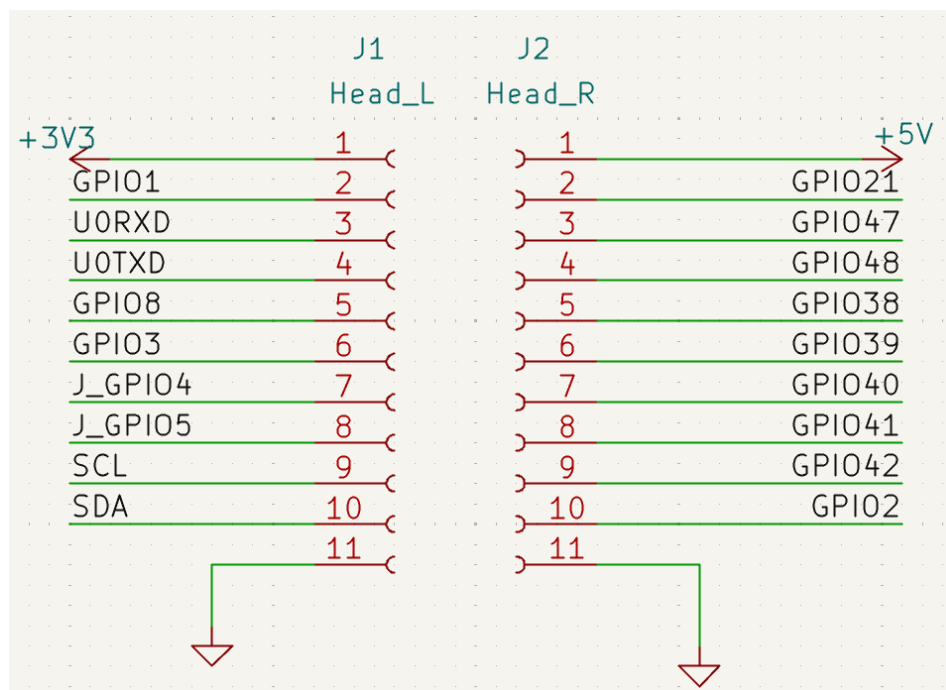
CONVERSIÓN ADC EN ESP32-S3

El ESP32-S3 incorpora 2 SAR ADC (Sucessive Approximation Registers, Analog-Digital Converters) de **12 bits** de resolución. Entre ambos permiten tener hasta 20 pines de entradas analógicas para el ESP32-S3.

Los canales ADC disponibles están asociados a los siguientes GPIO:

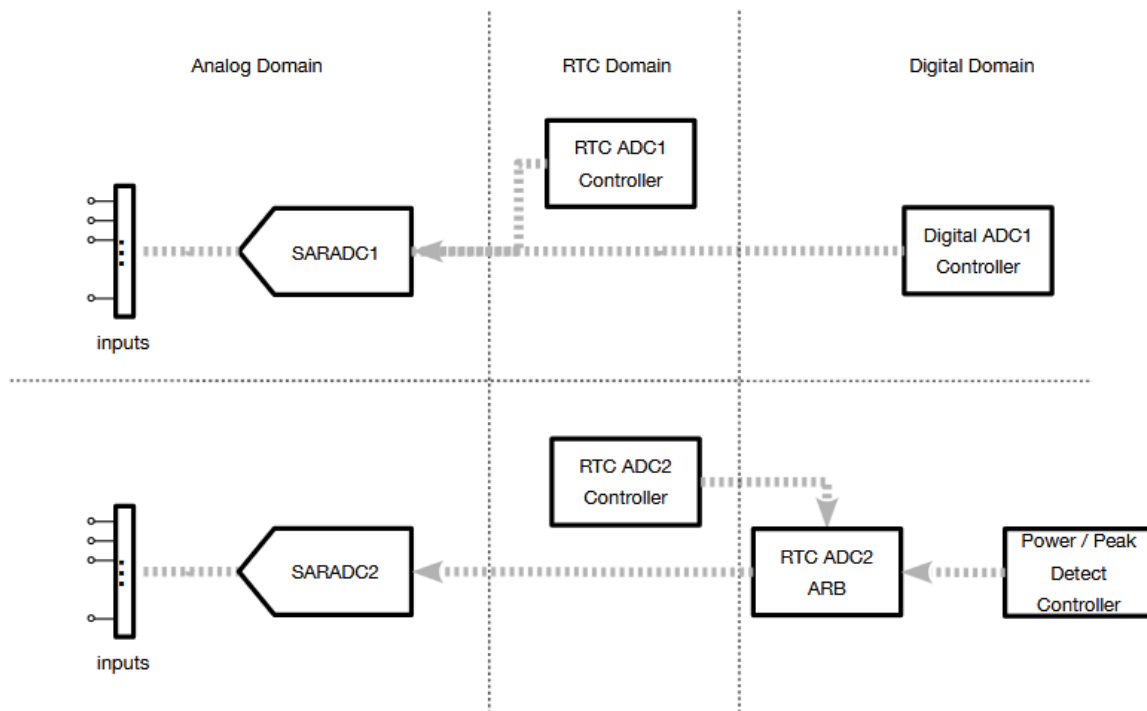
- **ADC1:** 10 canales (**GPIO1 a GPIO10**)
- **ADC2:** 10 canales (GPIO11 a GPIO20)

Sin embargo, es muy importante tener en cuenta esta limitación: **los canales del ADC2 NO pueden usarse si la WiFi está habilitada**. Teniendo en cuenta que en vuestro microbot la WiFi la vais a tener que usar, es mejor que te olvides del ADC2 para esta asignatura. Por otro lado, correlando los canales del ADC1 con los pines expuestos que te va a dejar la breadboard del microbot, que puedes recordar en esta Figura:



La conclusión es que vas a poder emplear cómodamente los siguientes canales del **ADC1: 1, 2, 3, 4, 5, 8.**

Los SAR ADC se manejan en el ESP32-S3 con cuatro controladores, como se muestra en esta Figura extraída del manual técnico.



- El Digital ADC1 se encarga de gestionar conversiones muy rápidas usando DMA y escaneo multicanal. En principio no lo vas a necesitar en la asignatura.
- El Power/Peak Detect Controller se usa para monitorizar la intensidad RF. Tampoco vas a necesitarlo.
- Los RTC ADC1 y ADC2 son los que se encargan de las conversiones únicas a demanda (conversiones one-shot que son las que, en principio, usarás) y de la gestión de los ADC en modos de bajo consumo.

La gestión de los ADC puede ser compleja en escenarios que demandan velocidades rápidas, conversiones multicanal, etc. En el siguiente enlace puedes ver la guía completa de programación de Espressif para el ADC del ESP32-S3: <https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32s3/api-reference/peripherals/adc.html>

Para que puedas configurar y emplear rápidamente el ADC1 para conversiones one-shot, en esta asignatura ponemos a tu disposición el código de ejemplo para PlatformIO *ESP32_S3_espidf_ADC_PWM*. En él, básicamente se siguen los siguientes pasos:

- Inicializar la unidad ADC1.

- Configurar los canales del ADC1 que vas a utilizar. En este caso se selecciona para esos canales un nivel de atenuación de 12 dB para que las entradas estén en el rango de 0 a 3,9 voltios, conforme a la lógica de 3v3 que usan las entradas del ESP32-S3. También se configura la resolución del conversor a su valor por defecto, que es el máximo permitido (en el caso de los SAR ADC del ESP32-S3, eso son 12 bits por muestra).
- Realizados estos pasos, se puede ordenar la lectura del dato de cada canal ADC con la función `adc_oneshot_read`.
- Borrar la unidad ADC1 antes de cerrar el programa.

NOTA: En este código de ejemplo no se realiza calibración de los canales ADC. Si quieres realizarla, consulta la guía de programación de Espressif referenciada. Por otro lado, si es necesario es posible construir una tabla de calibración propia para tu sensor, a mano, tal y como se explica en la asignatura.

GENERACIÓN DE PWM (para LEDs) EN ESP32-S3

El ESP32-S3 incorpora dos periféricos que sirven para generar PWM:

- LED Control (LEDC): <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/ledc.html>
- Motor Control Pulse Width Modulation (MCPWM): <https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32s3/api-reference/peripherals/mcpwm.html>

El LEDC permite generar PWM básicos de forma más sencilla: seleccionas la frecuencia, la resolución, el ciclo de trabajo, y generas una señal PWM. Es perfectamente posible usar LEDC para controlar un servo.

El MCPWM permite generar PWM más complejos, y más adecuados para el control de motores. Así, es posible un ajuste más fino del ciclo de trabajo, generar señales complementarias, insertar tiempos muertos, o recibir señales de sincronismo externas (SYNC) que permiten que la fase del PWM esté siempre correctamente alineada con la posición del motor.

El MCPWM lo veremos en la siguiente práctica, en la que lo usaremos para controlar los motores del microbot. Aquí vamos a explicar el (más sencillo) LEDC para que también tengas este recurso disponible. Sólo necesitas el código de ejemplo *ESP32_S3_espidf_ADC_PWM*.

En dicho código se incluye una función de configuración, `static void example_ledc_init(void)`, que básicamente:

1. Prepara primero el temporizador que gobierna el periférico LEDC,
2. y luego el canal de salida PWM.

Para la configuración del timer, siguiendo las instrucciones de la guía de programación de ESP-IDF, tienes que tener en cuenta que en el ESP32-S3 usarás **siempre** el reloj lento (`LEDC_LOW_SPEED_MODE`). También será normal dejar que el reloj se establezca de forma automática. Ten en cuenta que en el ESP32-S3 se utiliza una única fuente de reloj para todos los temporizadores, así que mejor no intentar forzar aquí una en concreto.

La resolución puedes dejarla en 13 bits, y ya la frecuencia y el ciclo de trabajo los haces corresponder con lo que necesites.

En cuanto al canal de salida PWM, lo más importante que tendrás que tener en cuenta es **indicar el GPIO por el que quieres que salga la señal**.

Una vez completada la configuración de temporizador y canal, con la función `ledc_set_duty` reconfiguras el ciclo de trabajo (pero no actualizas ese valor en el PWM), y con la función `ledc_update_duty` recargas el PWM para que, una vez acabe el ciclo actual, comience a ejecutar el nuevo ciclo. En la práctica llamarás a estas funciones una detrás de la otra. Como quiera que eso puede provocar problemas si hay, por ejemplo, varias tareas reconfigurando y recargando el mismo PWM a la vez, la API te proporciona `ledc_set_duty_and_update`, una función *thread-safe* que hace las dos cosas de una vez, evitando interferencias.

Recuerda que seleccionando una función y pulsando F12 en VSC vas a la definición de dicha función. En el caso del periférico LEDC merece la pena que le eches un vistazo a la API en `ledc.h` para ver las opciones adicionales que tienes disponibles para controlar este periférico, ya que son bastante sencillas de entender y directas de usar.